

Contents

Les Erreurs et Exceptions	3
1 Pourquoi gérer les erreurs ?	4
2 Comprendre une exception	4
3 Bloc try / except	4
4 Capturer l'objet exception	5
5 Plusieurs except	5
6 Capturer plusieurs exceptions ensemble	6
7 except Exception — usage contrôlé	6
8 La clause else	6
9 La clause finally	7
finally et return	7
Gestionnaire de contexte — with	8
1 1 Lever une exception avec raise	8
1 2 Relancer une exception	9
1 3 Chaînage d'exception	9
1 4 Exception personnalisée	9
1 5 Hiérarchie des exceptions	10
1 6 Assertion	11
1 7 contextlib.suppress — ignorer proprement	11
1 8 Propagation d'exception	11
1 9 Logging d'erreur	12
2 0 Bonnes pratiques	12
2 1 Cas réels	13
Saisie robuste	13
Lecture sécurisée	13
Flux complet structuré	14

2 2 Mini-exercices de production	14
Exercice 1 — Prédiction	14
Exercice 2 — Production	15
Exercice 3 — Débogage	16
Synthèse	17
20 Questions Intelligentes — Gestion des Erreurs	18
1) Pourquoi except: nu est dangereux ?	18
Énoncé	18
Solution	18
2) Différence entre Exception et BaseException	18
Énoncé	18
Solution	18
3) Ordre des blocs except	19
Énoncé	19
Solution	19
4) finally et return	20
Énoncé	20
Solution	20
5) finally exécuté avec exception non capturée ?	20
Énoncé	20
Solution	20
6) Relancer une exception	21
Énoncé	21
Solution	21
7) Différence entre raise e et raise	21
Énoncé	21
Solution	21
8) Exception personnalisée avec données	22
Énoncé	22
Solution	22
9) Propagation automatique	22
Énoncé	22
Solution	23
10) Assertion désactivable	23
Énoncé	23
Solution	23

11) Capture multiple en une ligne	24
Énoncé	24
Solution	24
12) Contexte with et exceptions	24
Énoncé	24
Solution	24
13) StopIteration explicite	25
Énoncé	25
Solution	25
14) Chaînage d'exception	25
Énoncé	25
Solution	25
15) try minimal	26
Énoncé	26
Solution	26
16) KeyError vs get()	26
Énoncé	26
Solution	27
17) Erreur silencieuse	27
Énoncé	27
Solution	27
18) Logging d'erreur	28
Énoncé	28
Solution	28
19) Retry contrôlé	28
Énoncé	28
Solution	28
20) Gestion propre d'un flux complet	29
Énoncé	29
Solution	29

Les Erreurs et Exceptions

Le support est propriété de **Bechir Bejaoui**

1 Pourquoi gérer les erreurs ?

Dans tout programme réel :

- Division par zéro
- Fichier introuvable
- Donnée utilisateur invalide
- Connexion réseau interrompue
- Clé inexistante dans un dictionnaire

Sans gestion d'erreur, le programme s'arrête brutalement.

Objectif :

- Rendre le code robuste
 - Contrôler le flux d'exécution
 - Fournir des messages explicites
 - Garantir une terminaison propre
-

2 Comprendre une exception

Une exception est un objet représentant une anomalie détectée à l'exécution.

```
"""  
Sans gestion, une exception arrête immédiatement le programme.  
"""
```

```
def division(a, b):  
    return a / b  
  
print(division(10, 0))    # ZeroDivisionError → arrêt brutal
```

3 Bloc try / except

```
"""  
try : contient le code risqué  
except : traite l'exception si elle survient  
Toujours capturer le type d'exception le plus précis possible.  
"""  
  
def division(a, b):
```

```

try:
    return a / b
except ZeroDivisionError:
    print("Division par zéro interdite")
    return None

print(division(10, 0))
print(division(10, 2))

```

4 Capturer l'objet exception

```

"""
Le mot clé as permet d'accéder à l'objet exception.
Il contient le message d'erreur et les attributs de l'exception.
"""

def division(a, b):
    try:
        return a / b
    except ZeroDivisionError as erreur:
        print("Erreur détectée :", erreur)
        return None

division(10, 0)

```

5 Plusieurs except

```

"""
Chaque exception est traitée séparément.
Toujours placer les exceptions spécifiques avant les générales.
"""

try:
    a = int(input("Nombre 1 : "))
    b = int(input("Nombre 2 : "))
    print(a / b)
except ValueError:
    print("Entrée invalide")
except ZeroDivisionError:
    print("Division par zéro")

```

6 Capturer plusieurs exceptions ensemble

```
"""
Un tuple d'exceptions permet de les capturer en une seule clause.
"""

try:
    int("abc")
except (ValueError, TypeError) as e:
    print("Erreur de conversion :", e)
```

7 except Exception — usage contrôlé

```
"""
Exception est la classe de base des exceptions courantes.
Elle n'inclut pas KeyboardInterrupt ni SystemExit.
Ne jamais utiliser except: nu - il capture TOUT.
"""

# Dangereux
try:
    x = 1 / 0
except:                                # capture KeyboardInterrupt et SystemExit
    print("Erreur capturée")

# Correct
try:
    x = 1 / 0
except Exception as e:                # exclut les interruptions système
    print("Erreur :", e)
```

8 La clause else

```
"""
else s'exécute uniquement si aucune exception n'a été levée.
Permet de séparer le code de succès du code de traitement d'erreur.
"""

def division(a, b):
    try:
        resultat = a / b
    except ZeroDivisionError:
```

```

        print("Erreur")
    else:
        print("Succès :", resultat)

division(10, 2)
division(10, 0)

```

9 La clause finally

```

"""
finally s'exécute TOUJOURS - qu'il y ait exception ou non.
Indispensable pour libérer les ressources.
"""

```

```

def ouvrir_fichier(nom):
    fichier = None
    try:
        fichier = open(nom, "r")
        print(fichier.read())
    except FileNotFoundError:
        print("Fichier introuvable")
    finally:
        if fichier:
            fichier.close()
        print("Bloc finally exécuté")

```

```

ouvrir_fichier("test.txt")

```

finally et return

```

"""
finally s'exécute même si try contient un return.
Un return dans finally écrase celui du try.
"""

```

```

def test():
    try:
        return "Try"
    finally:
        print("Finally exécuté")  # toujours affiché

print(test())  # "Try" - mais finally s'exécute avant

```

Gestionnaire de contexte — with

```
"""
with garantit la libération des ressources même en cas d'exception.
Équivalent propre du try/finally pour les ressources.
Plus lisible et plus sûr que la gestion manuelle.
"""

# Sans with - risqué
fichier = None
try:
    fichier = open("data.txt", "r")
    contenu = fichier.read()
finally:
    if fichier:
        fichier.close()

# Avec with - recommandé
try:
    with open("data.txt", "r") as f:
        contenu = f.read()
        print(contenu)
except FileNotFoundError:
    print("Fichier absent")
```

1 1 Lever une exception avec raise

```
"""
raise permet de signaler explicitement une condition d'erreur.
Utiliser les types d'exceptions standards si possible.
"""

def verifier_age(age: int) -> None:
    if age < 0:
        raise ValueError(f"Age invalide : {age}")
    print("Age valide :", age)

try:
    verifier_age(-5)
except ValueError as e:
    print("Erreur :", e)
```

1 2 Relancer une exception

```
"""
raise sans argument relance l'exception courante.
Conserve la pile d'appel originale.
raise e recrée la pile - perd l'origine exacte.
"""

try:
    try:
        1 / 0
    except ZeroDivisionError:
        print("Log erreur")
        raise # conserve la trace originale
except ZeroDivisionError:
    print("Propagation confirmée")
```

1 3 Chaînage d'exception

```
"""
raise ... from ... préserve l'exception d'origine.
Permet de tracer la cause racine de l'erreur.
"""

try:
    int("abc")
except ValueError as e:
    raise RuntimeError("Conversion échouée") from e
```

1 4 Exception personnalisée

```
"""
Créer ses propres exceptions pour les cas métier.
Hériter de Exception.
Enrichir avec des attributs métier.
"""

class SoldeInsuffisantError(Exception):
    def __init__(self, solde: float, montant: float):
        self.solde = solde
        self.montant = montant
        super().__init__(f"Solde {solde} € insuffisant pour retirer {montant} €")
```

```
def retirer(solde: float, montant: float) -> float:
    if montant > solde:
        raise SoldeInsuffisantError(solde, montant)
    return solde - montant

try:
    retirer(100, 200)
except SoldeInsuffisantError as e:
    print(e)
    print(f"Manque : {e.montant - e.solde} €")
```

1 5 Hiérarchie des exceptions

```
"""
BaseException
Exception
    ValueError
    TypeError
    ZeroDivisionError
    IndexError
    KeyError
    FileNotFoundError
    AttributeError

Capturer un parent capture aussi ses enfants.
"""

# Démonstration parent / enfant
try:
    1 / 0
except ArithmeticError:
    # parent de ZeroDivisionError
    print("ArithmeticError capturé")

# Vérification via isinstance
try:
    1 / 0
except Exception as e:
    print(isinstance(e, ZeroDivisionError))    # True
    print(isinstance(e, ArithmeticError))      # True
    print(isinstance(e, Exception))            # True
```

1 6 Assertion

```
"""
assert valide une précondition en développement.
Peut être désactivé avec python -O.
Ne jamais utiliser pour des validations métier en production.
"""

def moyenne(notes: list) -> float:
    assert len(notes) > 0, "Liste vide interdite"
    return sum(notes) / len(notes)

print(moyenne([10, 15]))
```

1 7 contextlib.suppress — ignorer proprement

```
"""
suppress ignore silencieusement une exception spécifique.
Plus expressif qu'un try/except pass.
"""

from contextlib import suppress

# À éviter
try:
    int("abc")
except ValueError:
    pass

# Préférer
with suppress(ValueError):
    int("abc")

print("Programme continue")
```

1 8 Propagation d'exception

```
"""
Si non capturée, l'exception remonte la pile d'appel.
Elle peut être capturée à n'importe quel niveau supérieur.
"""
```

```

def niveau2():
    raise ValueError("Erreur interne")

def niveau1():
    niveau2()

try:
    niveau1()
except ValueError as e:
    print("Exception capturée :", e)

```

1 9 Logging d'erreur

```

"""
logging est préférable à print pour les erreurs en production.
Il permet de configurer le niveau, le format et la destination.
"""

import logging

logging.basicConfig(level=logging.ERROR)

try:
    1 / 0
except ZeroDivisionError:
    logging.error("Division par zéro détectée", exc_info=True)

```

2 0 Bonnes pratiques

- Capturer uniquement ce que l'on peut traiter
- Ne jamais masquer les erreurs critiques avec `except: pass`
- Utiliser `with` pour gérer les ressources
- Nettoyer dans `finally` si `with` n'est pas possible
- Utiliser des exceptions personnalisées pour les cas métier
- Logger les erreurs plutôt que les afficher avec `print`
- Toujours placer les exceptions spécifiques avant les générales
- `raise` sans argument pour relancer — `raise e` perd la trace

2 1 Cas réels

Saisie robuste

```
"""
Boucle de saisie avec retry contrôlé.
"""

def saisir_entier(max_tentatives: int = 3) -> int:
    tentatives = 0
    while tentatives < max_tentatives:
        try:
            return int(input("Entrez un entier : "))
        except ValueError:
            tentatives += 1
            restantes = max_tentatives - tentatives
            print(f"Valeur incorrecte. Tentatives restantes : {restantes}")
    raise RuntimeError("Nombre maximum de tentatives atteint")

try:
    x = saisir_entier()
    print("Valeur valide :", x)
except RuntimeError as e:
    print("Abandon :", e)
```

Lecture sécurisée

```
"""
Lecture de fichier avec gestion propre des erreurs.
"""

def lire_fichier(nom: str) -> str:
    try:
        with open(nom, "r", encoding="utf-8") as f:
            return f.read()
    except FileNotFoundError:
        print(f"Fichier absent : {nom}")
        return ""
    except PermissionError:
        print(f"Accès refusé : {nom}")
        return ""
```

```
contenu = lire_fichier("data.txt")
```

Flux complet structuré

```
"""
Exemple complet combinant try, except, else, finally.
"""

class AuthentificationError(Exception):
    pass

def authentifier(login: str, password: str) -> bool:
    if login != "admin" or password != "1234":
        raise AuthentificationError("Identifiants invalides")
    return True

try:
    authentifier("admin", "0000")
except AuthentificationError as e:
    print("Erreur :", e)
else:
    print("Connexion réussie")
finally:
    print("Fin du processus")
```

2 2 Mini-exercices de production

Exercice 1 — Prédiction

Prédire la sortie avant exécution :

```
def test():
    try:
        return "Try"
    finally:
        print("Finally")

print(test())

try:
    try:
        1 / 0
    except ZeroDivisionError:
```

```

        print("Inner")
        raise
except ZeroDivisionError:
    print("Outer")
finally:
    print("Done")

```

Exercice 2 — Production

Écrire une fonction `convertir_nombre` qui :

1. Accepte une chaîne en entrée
2. Tente de la convertir en `int`, puis en `float` si échec
3. Lève une `ValueError` personnalisée si les deux échouent
4. Retourne la valeur convertie avec son type
5. Documente avec le format Google

Votre solution ici

Solution

```

"""
Conversion robuste avec gestion d'exceptions chaînées.
"""

class ConversionError(ValueError):
    def __init__(self, valeur: str):
        self.valeur = valeur
        super().__init__(f"Impossible de convertir '{valeur}' en nombre")

def convertir_nombre(chaine: str):
    """
    Convertit une chaîne en nombre.

    Args:
        chaine : Chaîne à convertir.

    Returns:
        Tuple (valeur convertie, type).

    Raises:
        ConversionError : Si la conversion est impossible.
    """
    try:
        valeur = int(chaine)
        return valeur, int

```

```

except ValueError:
    pass

try:
    valeur = float(chaine)
    return valeur, float
except ValueError:
    raise ConversionError(chaine)

for test in ["42", "3.14", "abc"]:
    try:
        valeur, type_val = convertir_nombre(test)
        print(f"'{test}' → {valeur} ({type_val.__name__})")
    except ConversionError as e:
        print(f"Erreur : {e}")

```

Exercice 3 — Débogage

Ce code contient 4 erreurs. Les identifier et les corriger.

```

try:
    x = 1 / 0
except:
    pass

try:
    int("abc")
except Exception:
    print("Erreur")
except ValueError:
    print("ValueError")

def retirer(solde, montant):
    if montant > solde:
        raise "Solde insuffisant"

try:
    int("abc")
except ValueError as e:
    raise RuntimeError("Conversion échouée")

```

Solution

```

# Erreur 1 : except: ne capture tout y compris KeyboardInterrupt
try:
    x = 1 / 0

```



```

except ZeroDivisionError:
    print("Division par zéro")

# Erreur 2 : Exception avant ValueError → ValueError jamais atteint
# Toujours placer le plus spécifique en premier
try:
    int("abc")
except ValueError:
    print("ValueError")
except Exception:
    print("Erreur générale")

# Erreur 3 : raise attend un objet exception - pas une chaîne
def retirer(solde, montant):
    if montant > solde:
        raise ValueError("Solde insuffisant")

# Erreur 4 : raise sans from perd la trace originale
try:
    int("abc")
except ValueError as e:
    raise RuntimeError("Conversion échouée") from e    # from e préserve la cause

```

Synthèse

- try/except capture et traite les exceptions
- else s'exécute uniquement si aucune exception n'est levée
- finally s'exécute toujours — même avec **return** ou exception non capturée
- with garantit la libération des ressources — préférer à try/finally manuel
- Toujours capturer le type d'exception le plus précis
- Ne jamais utiliser **except:** nu — utiliser au minimum **except Exception**
- **raise** sans argument relance en conservant la trace originale
- **raise ... from ...** chaîne les exceptions et préserve la cause racine
- Les exceptions personnalisées enrichissent les cas métier
- **contextlib.suppress** ignore proprement une exception attendue
- Logger les erreurs avec **logging** plutôt qu'afficher avec **print**

20 Questions Intelligentes — Gestion des Erreurs

1) Pourquoi `except:` nu est dangereux ?

Énoncé

Pourquoi éviter `except:` sans préciser le type d'exception ?

Solution

```
"""
Explication :
except: capture TOUT, y compris KeyboardInterrupt et SystemExit.
Cela peut bloquer l'arrêt normal du programme.
"""

# Dangereux
try:
    x = 1 / 0
except:
    print("Erreur capturée")      # capture aussi Ctrl+C

# Correct
try:
    x = 1 / 0
except ZeroDivisionError:
    print("Division par zéro")
```

2) Différence entre Exception et BaseException

Énoncé

Pourquoi capturer Exception plutôt que BaseException ?

Solution

```
"""
Explication :
BaseException inclut KeyboardInterrupt et SystemExit.
```

```
Exception les exclut - plus sûr pour les exceptions courantes.
"""
```

```
try:
    raise KeyboardInterrupt()
except Exception:
    print("Ne capture pas KeyboardInterrupt")
except BaseException:
    print("BaseException capture tout")
```

3) Ordre des blocs except

Énoncé

Pourquoi l'ordre des except est important ?

Solution

```
"""
Explication :
Les blocs sont évalués de haut en bas.
Un parent avant un enfant rend l'enfant inatteignable.
Toujours placer les exceptions spécifiques en premier.
"""
```

```
# Incorrect - ZeroDivisionError jamais atteint
```

```
try:
    1 / 0
except Exception:
    print("Parent capturé en premier")
except ZeroDivisionError:
    print("Jamais exécuté")
```

```
# Correct
```

```
try:
    1 / 0
except ZeroDivisionError:
    print("Spécifique d'abord")
except Exception:
    print("Général ensuite")
```

4) finally et return

Énoncé

Quel est le comportement de return dans finally ?

Solution

```
"""
Explication :
Un return dans finally écrase celui du try.
finally s'exécute toujours avant que la fonction retourne.
"""

def test_return():
    try:
        return "Try"
    finally:
        return "Finally"    # écrase "Try"

def test_print():
    try:
        return "Try"
    finally:
        print("Finally exécuté")    # s'exécute, ne remplace pas

print(test_return())    # Finally
print(test_print())    # Finally exécuté \n Try
```

5) finally exécuté avec exception non capturée ?

Énoncé

finally est-il exécuté si l'exception n'est pas attrapée ?

Solution

```
"""
Explication :
Oui, finally s'exécute même si l'exception remonte
sans être capturée dans le bloc courant.
"""

try:
```

```

    try:
        raise ValueError("Erreur")
    finally:
        print("Finally interne exécuté")    # toujours
except ValueError:
    print("Exception capturée au niveau supérieur")

```

6) Relancer une exception

Énoncé

Comment relancer une exception après traitement partiel ?

Solution

```

"""
Explication :
raise sans argument relance l'exception courante.
Conserve la pile d'appel originale intacte.
"""

try:
    try:
        1 / 0
    except ZeroDivisionError:
        print("Log erreur interne")
        raise    # relance avec trace originale
except ZeroDivisionError:
    print("Propagation confirmée")

```

7) Différence entre raise e et raise

Énoncé

Pourquoi raise seul est préférable ?

Solution

```

"""
Explication :
raise    → conserve la pile d'appel originale
raise e  → recrée la pile depuis ce point - perd l'origine exacte

```

```

"""

try:
    try:
        1 / 0
    except ZeroDivisionError as e:
        raise # trace originale conservée
except ZeroDivisionError:
    print("Trace originale préservée")

```

8) Exception personnalisée avec données

Énoncé

Comment enrichir une exception métier ?

Solution

```

"""
Explication :
Une exception peut contenir des attributs métier.
Appeler super().__init__() avec le message.
"""

class SoldeInsuffisantError(Exception):
    def __init__(self, solde: float, montant: float):
        self.solde = solde
        self.montant = montant
        super().__init__(f"Solde {solde} insuffisant pour {montant}")

try:
    raise SoldeInsuffisantError(100, 200)
except SoldeInsuffisantError as e:
    print(e)
    print("Solde   :", e.solde)
    print("Montant :", e.montant)

```

9) Propagation automatique

Énoncé

Pourquoi une exception traverse plusieurs niveaux ?

Solution

```
"""
Explication :
Si non capturée, l'exception remonte la pile d'appel.
Elle peut être capturée à n'importe quel niveau supérieur.
"""

def niveau2():
    raise ValueError("Erreur profonde")

def niveau1():
    niveau2()

try:
    niveau1()
except ValueError as e:
    print("Exception remontée :", e)
```

10) Assertion désactivable

Énoncé

Pourquoi assert ne doit pas remplacer une validation métier ?

Solution

```
"""
Explication :
Les assertions sont désactivées avec python -O.
Elles ne garantissent pas la validation en production.
Utiliser raise ValueError pour les validations métier.
"""

# Précondition développement uniquement
def verifier_dev(x):
    assert x > 0, "Valeur invalide"
    return x

# Validation métier - toujours active
def verifier_metier(x):
    if x <= 0:
        raise ValueError(f"Valeur invalide : {x}")
    return x
```

11) Capture multiple en une ligne

Énoncé

Comment capturer plusieurs exceptions ensemble ?

Solution

```
"""
Explication :
Un tuple d'exceptions dans except permet
de les traiter avec le même bloc.
"""

try:
    int("abc")
except (ValueError, TypeError) as e:
    print("Erreur de conversion :", e)
```

12) Contexte with et exceptions

Énoncé

Pourquoi préférer with pour les ressources ?

Solution

```
"""
Explication :
with garantit la fermeture même en cas d'exception.
Plus lisible et plus sûr que try/finally manuel.
"""

# Avec try/finally - verbeux
fichier = None
try:
    fichier = open("data.txt")
    data = fichier.read()
finally:
    if fichier:
        fichier.close()
```



```
# Avec with - recommandé
try:
    with open("data.txt") as f:
        data = f.read()
except FileNotFoundError:
    print("Fichier absent")
```

13) StopIteration explicite

Énoncé

Pourquoi StopIteration ne doit pas être levée dans un générateur ?

Solution

```
"""
Explication :
Depuis Python 3.7, lever StopIteration dans un générateur
provoque RuntimeError - comportement indéfini avant.
Utiliser return pour terminer un générateur.
"""

def gen_correct():
    yield 1
    yield 2
    return # termine proprement

print(list(gen_correct()))
```

14) Chaînage d'exception

Énoncé

Comment préserver l'exception d'origine ?

Solution

```
"""
Explication :
raise ... from ... chaîne les exceptions.
Permet de tracer la cause racine tout en levant une nouvelle exception.
"""
```

```
try:
    int("abc")
except ValueError as e:
    raise RuntimeError("Conversion échouée") from e
```

15) try minimal

Énoncé

Pourquoi limiter le code dans try ?

Solution

```
"""
Explication :
Un try trop large masque l'origine réelle de l'exception.
Ne placer que la ligne risquée dans le try.
"""

# Trop large - quelle ligne provoque l'erreur ?
try:
    a = int(input("a : "))
    b = int(input("b : "))
    r = a / b
    print(r)
except Exception:
    print("Erreur quelque part")

# Précis - chaque risque identifié
try:
    a = int(input("a : "))
except ValueError:
    print("Valeur a invalide")
```

16) KeyError vs get()

Énoncé

Pourquoi préférer dict.get ?

Solution

```
"""
Explication :
L'accès direct lève KeyError si la clé est absente.
get() retourne une valeur par défaut - plus sûr.
"""

data = {"a": 1}

try:
    print(data["b"])
except KeyError:
    print("Clé absente")

print(data.get("b", 0))    # 0 - sans exception
```

17) Erreur silencieuse

Énoncé

Pourquoi ne rien faire dans except est dangereux ?

Solution

```
"""
Explication :
Un except vide masque le bug.
L'erreur reste présente mais invisible.
Toujours au minimum logger l'exception.
"""

import logging

# Dangereux
try:
    1 / 0
except ZeroDivisionError:
    pass    # bug masqué

# Correct
try:
    1 / 0
except ZeroDivisionError:
```

```
logging.error("Division par zéro détectée", exc_info=True)
```

18) Logging d'erreur

Énoncé

Pourquoi logger plutôt que print ?

Solution

```
"""
Explication :
logging permet de configurer niveau, format et destination.
exc_info=True ajoute la stack trace complète.
Indispensable en production.
"""

import logging

logging.basicConfig(
    level=logging.ERROR,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

try:
    1 / 0
except ZeroDivisionError:
    logging.error("Division par zéro détectée", exc_info=True)
```

19) Retry contrôlé

Énoncé

Comment implémenter un retry simple ?

Solution

```
"""
Explication :
Boucle avec compteur de tentatives.
Abandon après le nombre maximum atteint.
"""
```

```

MAX = 3

for tentative in range(1, MAX + 1):
    try:
        x = int(input(f"Tentative {tentative}/{MAX} - Nombre : "))
        print("Valide :", x)
        break
    except ValueError:
        print("Valeur incorrecte")
else:
    print("Abandon après", MAX, "tentatives")

```

20) Gestion propre d'un flux complet

Énoncé

Construire un flux robuste combinant try, except, else, finally.

Solution

```

"""
Explication :
Combinaison complète de tous les blocs.
try      → code risqué
except   → traitement de l'erreur
else     → exécuté si aucune exception
finally  → exécuté dans tous les cas
"""

class OperationError(Exception):
    pass

def operation(a: float, b: float) -> float:
    if b == 0:
        raise OperationError("Division interdite")
    return a / b

try:
    resultat = operation(10, 2)
except OperationError as e:
    print("Erreur :", e)
else:
    print("Succès :", resultat)

```

```
finally:  
    print("Fin du traitement")
```